Predicting if Customers Subscribe to a Term Deposit

1. Business Understanding

The primary business objective is to develop a predictive model that can accurately classify whether a bank client will subscribe to a term deposit (the target variable, `y`) based on their demographic information, past interactions, and current economic indicators.

By successfully predicting this outcome, the bank can:

1. Optimize Marketing Campaigns: Targeting customers who are most likely to subscribe and reduce the time and cost spent on calling unlikely prospects.
2. Resource Allocation: Improving the efficiency of the telemarketing team.
3. Increase Conversion Rates: Ultimately driving more revenue by focusing efforts on high-probability leads.

2. Data Understanding

The data was collected from a Portuguese retail bank over a span of five years from May 2008 to June 2013 for a total of 52944 phone contacts. Based on the df.info() output (which shows non-null counts) and the provided data description, here's an assessment of potential missing values and data type considerations:

2.1 Missing Values:
Although df.info() may indicate complete columns with no NaN values, some entries are designated as missing or need special processing according to the data description.

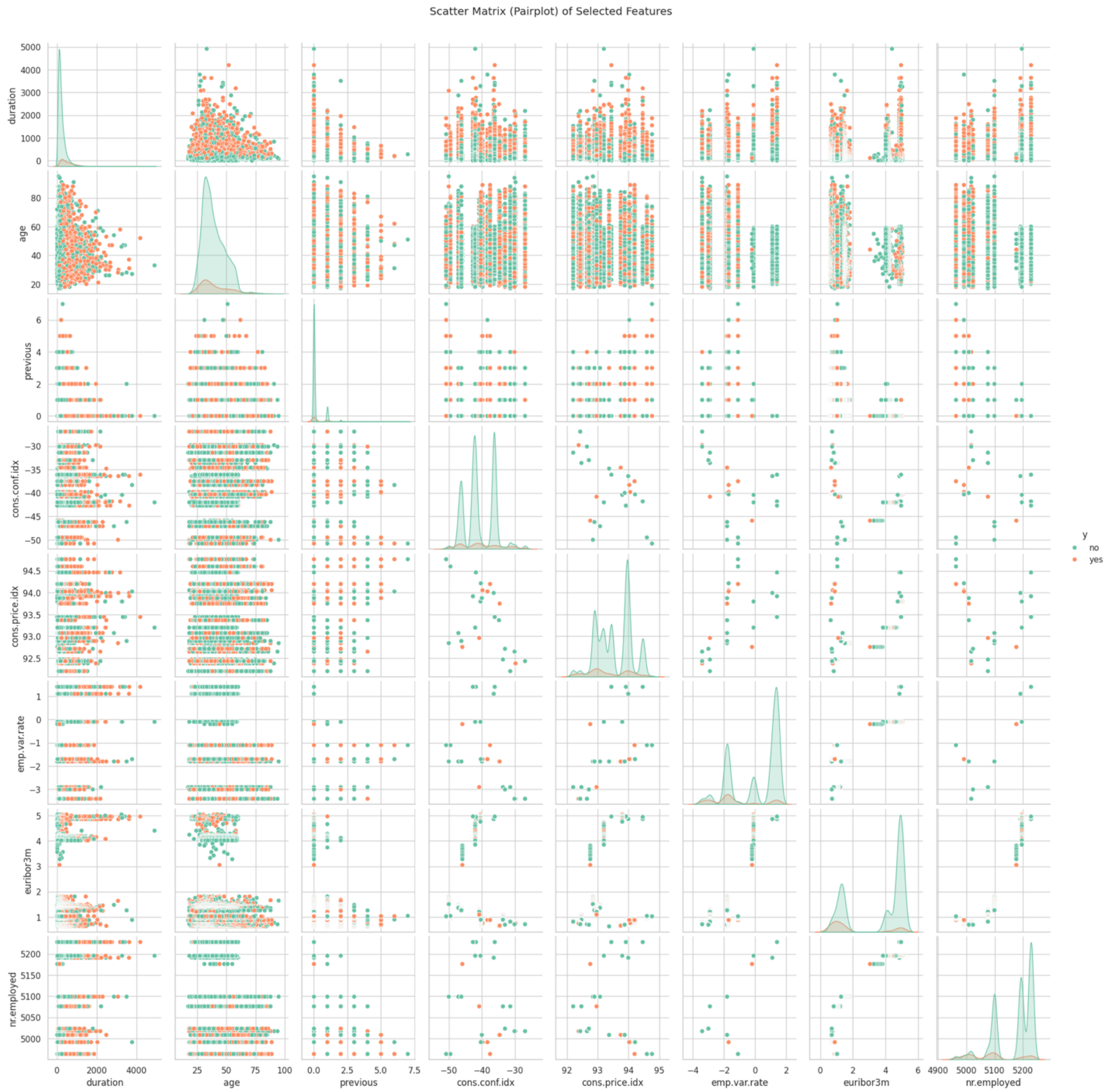
2.2 Categorical features with 'unknown': The job, marital, education, default, housing, and loan columns all include 'unknown' as one of the possible categories. These 'unknown' values should be treated as missing data for modeling purposes and may require imputation or removal.
pdays (numeric): A value of 999 means the client was not previously contacted. This is a sentinel for missing data and can be converted to NaN or used as a binary feature.

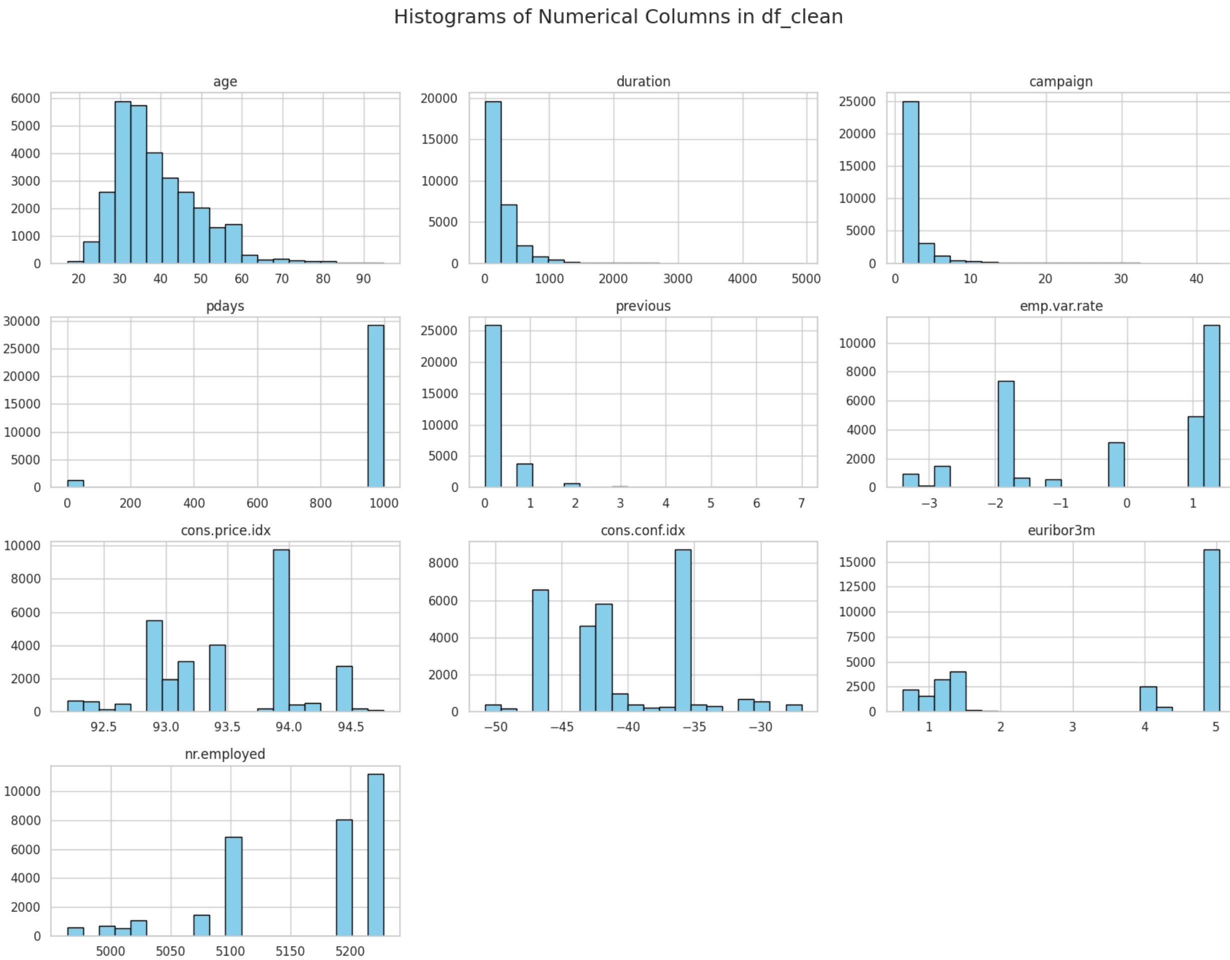
2.3 Data Type Coercion:
Pandas loads all columns with appropriate data types (int64 for numerics, object for categorical strings). Categorical columns (job, marital, education, default, housing, loan, contact, month, day_of_week, poutcome, y) must be encoded into numeric values for modeling. The binary y column ('yes'/'no') should also be converted to numeric (0/1).

2.4 Other Considerations:
duration (numeric): Since this attribute is only available after the call and strongly affects the target, it should be excluded from realistic predictive models. Keep it only for benchmarking purposes.

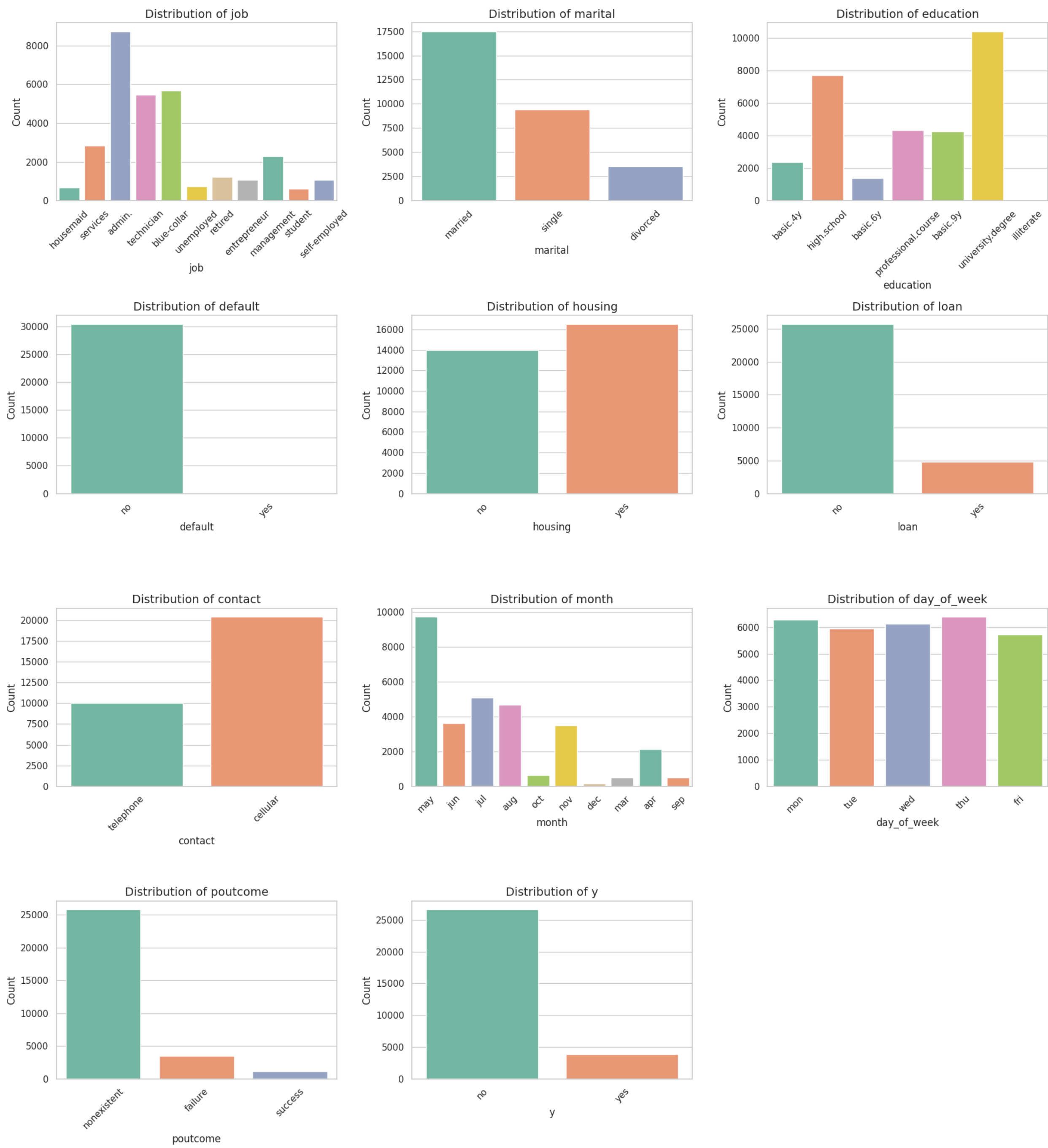
2.5 Data Visualization and Insights
Pairplots clearly show that data is imbalanced with “no” outcomes far outweighing “yes”. Visually inspecting pairplots did not reveal that any feature can naturally separate “yes” and “no”. Furthermore, except for age and euribor rate, the other numeric data are discrete with limited number of values. As such data presentation in decision space for those features would be highly clustered.
Histograms show that data is well-behaved with no apparent outliers.

Count plots reveal interesting categorical features. For example, most calls were carried over cell phone, during month of May, to married individuals with an administrative job who also had a university degree.




Histograms of Numerical Columns in df_clean
age
0
1000
2000
3000
4000
5000
6000
20
30
40
50
60
70
80
90
duration
0
5000
10000
15000
20000
0
1000
2000
3000
4000
5000
campaign
0
5000
10000
15000
20000
25000
0
10
20
30
40
pdays
0
5000
10000
15000
20000
25000
30000
0
200
400
600
800
1000
previous
0
5000
10000
15000
20000
25000
0
1
2
3
4
5
6
7
emp.var.rate
0
2000
4000
6000
8000
10000
−3
−2
−1
0
1
cons.price.idx
0
2000
4000
6000
8000
10000
92.5
93.0
93.5
94.0
94.5
cons.conf.idx
0
2000
4000
6000
8000
−50
−45
−40
−35
−30
euribor3m
0
2500
5000
7500
10000
12500
15000
1
2
3
4
5
nr.employed
0
2000
4000
6000
8000
10000
5000
5050
5100
5150
5200

Count Plots of Non-Numerical Columns in df_clean



3. Data Preparation

The dataset was filtered to completely remove any rows that contained the exact string 'unknown' in any of their columns.
Original Dataset Size: 41,188 rows
Rows Removed: 10,700 rows
Cleaned Dataset Size: 30,488 rows
Data Retention: 74.02% of the original data was retained for further analysis and modeling.
One hot encoding was used for all categorical data except for outcome (yes or no), which was binary coded.

4. Modeling

4.1 Baseline Accuracy:

The baseline accuracy for this dataset is **87.43%**. Ignoring all customer features and simply predicted "No" (the majority class) for every single call, would be correct 87.43% of the time.

This serves as the absolute minimum threshold that other machine learning classifiers need to beat.

4.2 The Trap of Imbalanced Data:

Because the baseline is so imbalanced, relying solely on "Accuracy" can be highly misleading. A model could achieve an 88% accuracy by almost always predicting "No", but it would completely fail the business objective of identifying the actual "Yes" customers. This highlights why we must evaluate our models using metrics that are robust to class imbalance, such as **ROC AUC**.

4.3 Linear Regression:

Accuracy (0.8857): It is slightly better than our baseline accuracy (0.8743), meaning it's doing a little better than just guessing 'no' every time.

Precision (0.6777): When the model predicts someone will subscribe to a term deposit, it is correct about 68% of the time.

Recall (0.2330): This is quite low. The model only successfully identifies 23% of the people who actually end up subscribing.

ROC AUC (0.7972) & The ROC Curve: Given the data is highly imbalanced, this is the most appropriate evaluation metric. The ROC curve plots the True Positive Rate against the False Positive Rate at various decision threshold settings.

The plotted curve bows significantly towards the top-left corner, pulling far away from the dotted diagonal line (which represents random guessing).

An AUC (Area Under the Curve) of ~0.80 means there is an 80% chance that the model will assign a higher predicted probability to a randomly chosen positive instance (a true 'Yes') than to a randomly chosen negative instance (a true 'No'). This proves the model has successfully learned the strong underlying global linear trends (like the economic indicators) to effectively separate the two classes

4.4 Model Comparison:

	Model	Train Time	Train Accuracy	Test Accuracy	ROC AUC
0	Logistic Regression	0.930949	0.889709	0.885700	0.797201
1	KNN	0.040119	0.905986	0.871597	0.725269
2	Decision Tree	0.200130	0.994383	0.823221	0.625884
3	SVM	65.704371	0.900492	0.881109	0.711065

Logistic Regression is currently performing the best overall, achieving the highest ROC AUC (0.80) and test accuracy (0.89), while being extremely fast to train.

Decision Tree is severely overfitting the data. It has nearly 100% training accuracy but the lowest test accuracy and ROC AUC (~0.63).

SVM took significantly longer to train than the others (around 52 seconds) but didn't beat Logistic Regression in performance.

KNN is very fast but its ROC AUC is lower than Logistic Regression.

4.5 Hyperparameter Tuning:

	Model	Train Time	Train Accuracy	Test Accuracy	ROC AUC
0	Logistic Regression	0.930949	0.889709	0.885700	0.797201
1	Tuned Logistic Regression	1086.976308	0.889463	0.886028	0.796211
2	KNN	0.040119	0.905986	0.871597	0.725269
3	Tuned KNN	342.684988	0.896023	0.878485	0.752342
4	Decision Tree	0.200130	0.994383	0.823221	0.625884
5	Tuned Decision Tree	24.406133	0.892620	0.886356	0.770768
6	SVM	65.704371	0.900492	0.881109	0.711065
7	Tuned SVM	1816.893008	0.900492	0.881109	0.711065

Logistic Regression

Performed a Grid Search for the Logistic Regression model. It tested different regularization parameters (C), penalty types (L1 vs L2), and solvers that support these penalties.

It took about 1086 seconds (roughly 18 minutes). The Grid Search found the best parameters to be C=0.1, penalty='l1', and solver='saga'. The test set ROC AUC score for the best model is 0.7962.

The Tuned Logistic Regression achieved a Train Accuracy of ~88.95% and a Test Accuracy of ~88.60%. This is close to the baseline Logistic Regression, meaning the default parameters were already performing quite well.

Decision Tree

Tuning the hyperparameters (max_depth=5, min_samples_leaf=10, min_samples_split=2) completely eliminated the severe overfitting from the baseline model (which had ~99.4% train accuracy but only ~62.6% test ROC AUC). The tuned model generalizes much better to the unseen test data.

SVM

Even with the optimizations (subsampling and removing probability calculations), it took about 29 minutes (1752 seconds) to find the best parameters, and then another 64 seconds to train on the full dataset.

The Grid Search found that the best parameters from our grid are C=1 and kernel='rbf', which are actually the default parameters for SVC in scikit-learn. Consequently, the Tuned SVM has the exact same Train Accuracy (90.05%) and Test Accuracy (88.11%) as the baseline SVM model.

KNN

Tuning for the KNN model searches for the optimal number of neighbors, weight function, and distance metric. The results are then appended to the existing comparison DataFrame.

The Test Accuracy improved slightly from 87.16% to 87.85%. The Train Accuracy actually dropped slightly from 90.60% to 89.60%. It means the tuned model is less overfitted to the training data and generalizes better to unseen data.

The ROC AUC saw a solid improvement, jumping from 0.7253 to 0.7523. This indicates the tuned model is noticeably better at distinguishing between clients who will subscribe and those who won't.

The baseline KNN was almost instant to train (0.07 seconds). The Tuned KNN took much longer (324.6 seconds, or about 5.5 minutes) because it had to test many different combinations of hyperparameters (n_neighbors, weights, and p) using 5-fold cross-validation.

The grid search successfully found better hyperparameters (n_neighbors: 11, p: 1 [Manhattan distance], and weights: 'uniform') that sacrificed a tiny bit of training accuracy to achieve better generalization, a higher test accuracy, and a significantly improved ROC AUC score.

5. Evaluation

The Logistic regression showed the highest score for ROC-AUC. We use this model to generate recommendations. Below, pairwise decision boundaries and test samples are shown for four most important features. The features are selected based on ranked values of their absolute weight.

Top 4 features by absolute weight in Logistic Regression are:

1. emp.var.rate (weight: -2.3798)
2. cons.price.idx (weight: 1.2362)
3. nr.employed (weight: 0.5368)

4. contact_telephone (weight: -0.3378)

Given above, here are the recommendations:

- The employment rate has the strongest negative influence on whether a customer accepts a term deposit. When employment rate changes by more than +0.5%, the likelihood of customers accepting an offer drops sharply, making any marketing campaign during that period largely ineffective.
- Consumer price index is positively correlated with bank activity, as higher inflation prompts customers to use their savings more actively. In this environment, market campaigns are likely to be most effective.
- Likewise, an increased employment rate fosters greater savings. This is logical, as higher employment signifies enhanced economic stability, which in turn promotes saving behavior.
- Consumers generally respond unfavorably when contacted through landlines, so marketing efforts using them should be refrained from.